



DandelionDB

API Design Specification

Embedded Vector Database for Edge Devices

Direct Query Execution

This DB has effectively 3 main functions for execution. Create database, execute query, close database.

Others are either maintenance or not very important.

Create database

```
db = liondb::create("./file.lion");
```

```
db = liondb::open("./file.lion");
```

Errors On Create

1. parent directory doesn't exist
2. permission denied
3. file already exists
4. disk error
5. invalid database path

Errors On Open

1. file doesn't exist
2. permission denied
3. invalid/corrupted database
4. unsupported database version

Execute Query

```
result = db.execute("*query"?;
```

Result output type

```
pub enum QueryResult {  
    Empty,  
  
    Affected {  
        rows_affected: u64,  
        last_insert_id: Option<Value>,  
    },  
  
    Rows {  
        columns: Vec<Column>,  
        rows: Vec<Row>,  
    },  
}
```

Exact meaning of each field

QueryResult::Empty

Used for:

```
CREATE COLLECTION  
DROP COLLECTION  
CREATE INDEX  
DROP INDEX
```

No data is returned.

```
QueryResult::Affected {  
    rows_affected: u64,  
    last_insert_id: Option<Value>,  
}
```

Used for:

INSERT
UPDATE
DELETE

Rows_affected: u64

Examples:

```
QueryResult::Affected {  
    rows_affected: 1,  
    last_insert_id: Some(Value::UInt(42)),  
}
```

For UPDATE:

```
QueryResult::Affected {  
    rows_affected: 5,  
    last_insert_id: None,  
}
```

For DELETE:

```
QueryResult::Affected {  
    rows_affected: 3,  
    last_insert_id: None,  
}
```

```
QueryResult::Rows {  
    columns: Vec<Column>,  
    rows: Vec<Row>,  
}
```

Used for:

FIND

Example:

```
QueryResult::Rows {  
    columns: vec![  
        Column {  
            name: "doc_id".to_string(),  
            data_type: DataType::UInt64,  
        },  
    ],  
    rows: Vec::new(),  
}
```

```

        Column {
            name: "name".to_string(),
            data_type: DataType::String,
        },
    ],

    rows: vec![
        Row {
            values: vec![
                Value::UInt(1),
                Value::String("Alice".to_string()),
            ],
        },
    ],
}

```

Close File

```
db.close("./file.lion"?;
```

Close Error

Error	When it can happen
I/O error	OS fails while writing pending data
Flush failed	Buffered database data cannot be flushed to the file
Sync failed	fsync /durability operation fails
Lock release failed	Database/file lock cannot be released correctly
Storage/device error	Disk becomes unavailable, filesystem error, etc.